M1: Machine learning - Coursework Assignment

Xueqing Xu (xx823)
Department of Physics, University of Cambridge
December 16, 2024

Word Count: 2062

Introduction

This project focuses on developing a machine-learning system to add two handwritten MNIST digits. I have created a complete pipeline that takes two digits as a combined 56×28 pixel image and predicts their sum in Pytorch. Later on, we compared the model performance with the classical statistical methods.

Implementation Details

1. Development Environment
   This project was developed using Python 3.10 within a conda environment on macOS, using Apple Silicon optimization through PyTorch's MPS (Metal Performance Shaders) backend for efficient computation.

2. Software Practices
   The development process followed industry-standard software engineering practices, using Git for version control with conventional commit messages. The codebase is organized into a modular structure, separating into distinct directories for dataset creation, model architectures, configurations, utility functions, and experiment results.

3. Reproducibility Guidelines
   To ensure reproducibility, the project implements a comprehensive environment management system using conda, with all dependencies specified in a requirement.txt file. The training pipeline is designed to be device-agnostic, automatically selecting between MPS, CUDA, and CPU based on hardware availability. Consistency across experiments is maintained through fixed random seeds for all probabilistic operations in numpy, torch, and the random library.

Methodology

Dataset Creation

The first step of this project is to create a custom MNIST addition dataset implemented through the MNISTAdditionDataset class, which inherits from PyTorch's Dataset class. The dataset is designed to combine pairs of MNIST digits into single images of dimension 56×28 pixels, where digits are concatenated horizontally. To ensure robust statistical properties, the implementation employs a balanced sampling strategy, generating 1,000 pairs for each possible sum (0-18) in the training set and 200 pairs per sum in the testing set, totaling 19,000 and 3,800 pairs respectively. Later on, we will train two models using different datasets and compare their performance.

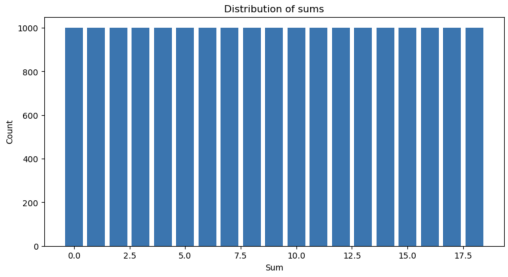

Figure 1: Balanced Dataset Distribution

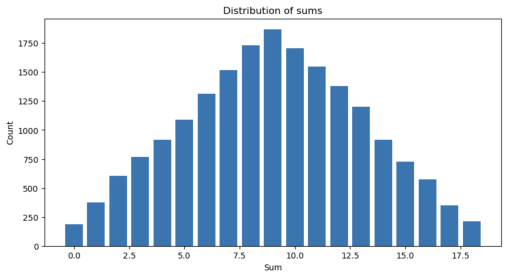

Figure 2: Imbalanced Dataset Distribution

The data pipeline includes normalization transforms (mean=0.1307, std=0.3081) and implements a deterministic random seed for reproducibility. The complete dataset is then split into training, validation, and test sets with ratio shown below. Then, the dataset is wrapped in PyTorch DataLoader objects with configurable batch sizes and memory optimization through pin memory.

Split	**Size**	**Percentage**
Training	16,150	70.8%
Validation	2,850	12.5%
Test	3,800	16.7%
Total	22,800	100%

Table 1: Dataset Split Distribution

Two other dataset creation pipelines were implemented to accommodate different model requirements and analysis approaches.

The first pipeline `prepare_classification_dataset.py` was designed to support single image classification by preserving individual digit labels alongside their sums. This implementation stores the left digit, right digit, and sum labels separately.

The second pipeline `prepare_sklearn_dataset.py` was optimized for compatibility with scikit-learn's classical machine learning algorithms. This version simplifies the data structure by storing only the combined images and their corresponding sums, conforming to scikit-learn's expected input format.

Both pipelines maintain the balanced sampling approach and implement identical normalization procedures (scaling pixel values to [0,1]), ensuring consistent statistical properties across different modeling approaches.

Model Architecture

The neural network architecture is a fully connected network (MNISTAdditionNN) designed for the digit addition task. The network takes 56×28 pixel images (1,568 dimensions) as input and processes them through multiple fully connected layers.

The architecture begins with an input layer followed by multiple hidden layers with consistent hidden size. Each hidden layer is followed by ReLU activation and dropout (rate=0.2) for regularization. The final layer outputs 19-dimensional logits corresponding to possible sums (0-18). The model's hyperparameters, including hidden size, number of layers, dropout rate, learning rate and batch size.

Training Procedure

The model was trained using the Adam optimizer with CrossEntropyLoss as the objective function. Training utilized early stopping with a patience of 5 epochs and a minimum delta of 0.001 to prevent overfitting. The default number of epoch is 50. During training, multiple metrics including loss, accuracy, and recall were tracked for both training and validation sets. The best performing model was saved based on validation loss, and comprehensive metrics were logged for each epoch. To ensure reproducibility, all random seeds were fixed and model checkpoints were saved regularly.

Results and Evaluation

Dataset Comparison

We evaluated the impact of dataset balancing strategies on model performance for the MNIST addition task. Two approaches were implemented and compared: a balanced dataset ensuring equal representation of sum classes (0-18), and an unbalanced dataset maintaining natural digit pair distributions.

Training Dynamics

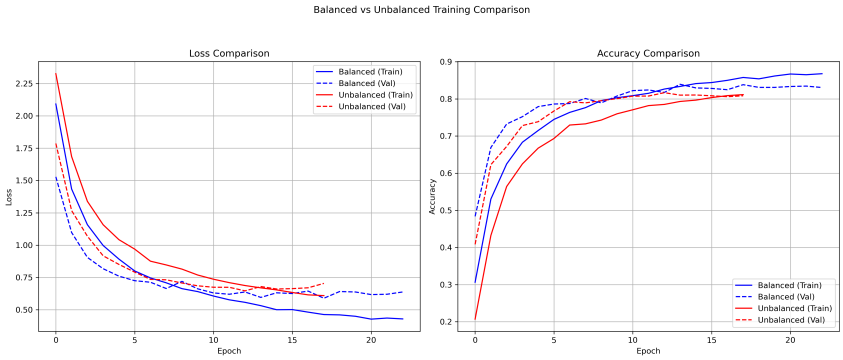

Figure 3: Training Summary for Imbalanced Dataset and Balanced Dataset

The training curves (Figure 3) reveal distinct characteristics between the two approaches. The model trained on balanced data achieved a lower final training loss than the unbalanced training. Additionally, validation accuracy curves shows more stable convergence for balanced training, suggesting more uniform learning across all sum classes. We are using an early stopping criterion here, so two models were trained using different number of epoches.

Model Performance

Table 2 presents the performance metrics on both balanced and unbalanced test sets. On the balanced test set, both models achieved comparable accuracy (83%), indicating that dataset balancing does not significantly impact overall performance when evaluated on evenly distributed data. However, when tested on an unbalanced dataset reflecting natural digit pair frequencies, the model trained on unbalanced data showed marginally better performance (82.89% vs 81.18%). Hence, we will train NN throughout the following comparison using an unbalanced dataset.

Performance on Unbalanced Test Dataset			
Model	Accuracy	Correct	Total
Balanced	0.8118	3085	3800
Unbalanced	0.8289	3150	3800
Performance on Balanced Test Dataset			
Model	Accuracy	Correct	Total
Balanced	0.8316	3160	3800
Unbalanced	0.8303	3155	3800

Table 2: Model Performance Comparison on Balanced and Unbalanced Test Sets

Hyperparameter FineTuning

We used Optuna, a hyperparameter optimization framework, to systematically explore the model's hyperparameter space. The optimization process involved 50 trials (running approximately 5 hours), each evaluating different combinations of five key hyperparameters:

- Hidden layer size (32-512 neurons)
- Number of layers (1-5)
- Dropout rate (0.1-0.5)
- Learning rate (1e-5 to 1e-2)
- Batch size (32, 64, 128, 256)

Implementation Analysis

The hyperparameter optimization process used a few efficiency mechanisms and a multi-stage evaluation approach. We implemented an early stopping strategy with a patience value of 3, meaning training would halt if no improvement in validation accuracy was observed over three consecutive epochs. This approach helped conserve computational resources by preventing unnecessarily long training sessions for suboptimal configurations. In total, Optuna completed 50 total trials.

The evaluation process followed a two-stage approach:

- During optimization, validation accuracy was used as the primary metric for comparing different hyperparameter configurations.
- After identifying the best-performing models based on validation performance, we conducted a final evaluation using a separate test dataset to assess true generalization capability.

Finetuned Result

Category	Parameter	Value/Range
Experiment Details	Total Experiments	50
	Best Accuracy	0.8698
	Best Experiment ID	11.0
Best Configuration	Hidden Size	384.0
	Number of Hidden Layers	4.0
	Dropout Rate	0.2570
	Learning Rate	$5.148e^{-4}$
	Batch Size	256.0
	Epochs Completed	26.0

Table 3: Hyperparameter Tuning Summary

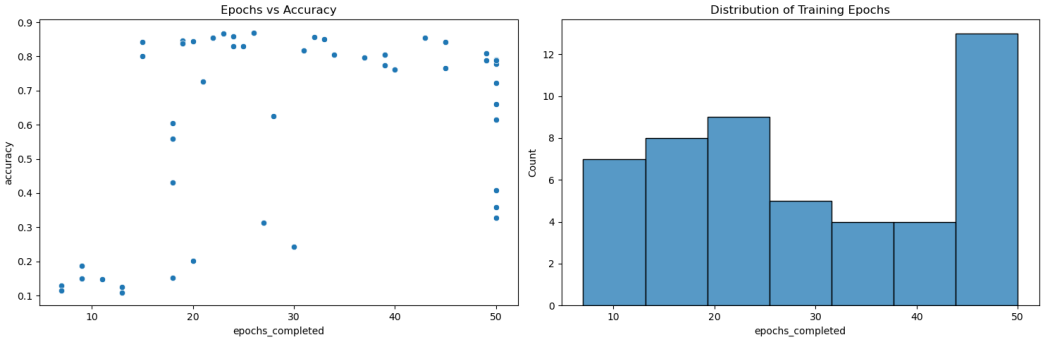

Figure 4: Epoch Analysis of Fine-tuned Results

The hyperparameter optimization process in Table 3 revealed promising results, with the best-performing model (trial 11) achieving a validation accuracy of 0.8698. This model used a moderately deep architecture with 4 hidden layers and 384 neurons per layer, complemented by a balanced dropout rate of 0.2570 for regularization. The optimization benefited from a careful learning rate of $5.148e{-}4$ and a relatively large batch size of 256.

Analysis of training dynamics, as shown in Figure 4, demonstrates that most successful models converged between 15-25 epochs, with peak performance typically achieved in the 20-30 epoch range. Interestingly, the distribution of training durations showed a bimodal pattern, with clusters around 20 and 50 epochs, though longer training times did not necessarily correlate with improved performance.

Test Performance

Here, we ran inference on these 50 best model that we stored in each trial using an unbalanced test dataset. Similarly, we found that the best model in trial 11 is the best-performing model, with a test accuracy of 0.8716.

The test performance analysis across different hyperparameters reveals several significant patterns in model behavior. The distribution of test accuracies (Figure 5) shows a bimodal pattern, with a large cluster of high-performing models achieving accuracies between 0.8-0.9, and a smaller group of poorly performing models with accuracies around 0.1-0.2.

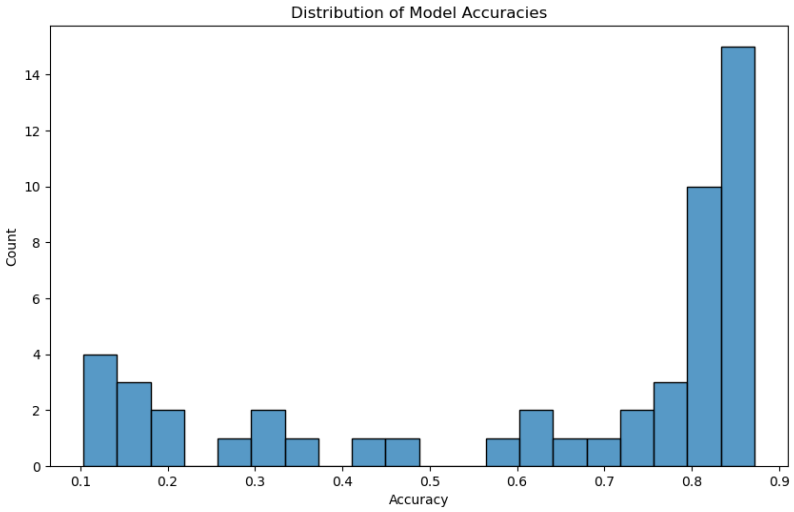

Figure 5: Distribution of Test Accuracy of 50 models

Examining the individual hyperparameter relationships in Figure 6, the hidden size plot demonstrates that models with 200-400 neurons generally achieved the best performance. The number of layers plot shows that models with 2-3 layers performed most consistently, while deeper networks (5 layers) showed more variable results with some models failing to learn effectively. The dropout rate analysis indicates optimal performance in the 0.2-0.3 range, suggesting that moderate regularization was beneficial for generalization. Learning rate proved to be a critical parameter, with the best performance consistently achieved with rates around 0.001, and a sharp decline in performance for both very small and large learning rates. The batch size relationship shows relatively consistent performance across different sizes.

Best Performing Model

Here we will summarise and visualize the range of weights of each layer in the best-performing model found in trial 11.

Layer	**Shape**	**Parameters**	**Mean**	**Std**	**Range**
hidden layer 1.weight	(384, 1568)	602,112	0.000694	0.025017	[-0.243, 0.165]
hidden layer 2.weight	(384, 384)	147,456	-0.004302	0.043036	[-0.244, 0.165]
hidden layer 3.weight	(384, 384)	147,456	-0.002612	0.044585	[-0.223, 0.192]
hidden layer 4.weight	(384, 384)	147,456	-0.000268	0.042427	[-0.222, 0.210]
output layer.weight	(19, 384)	7,296	-0.014925	0.054369	[-0.372, 0.139]
Total Parameters		1,051,776			

Table 4: Neural Network Layer Weight Statistics

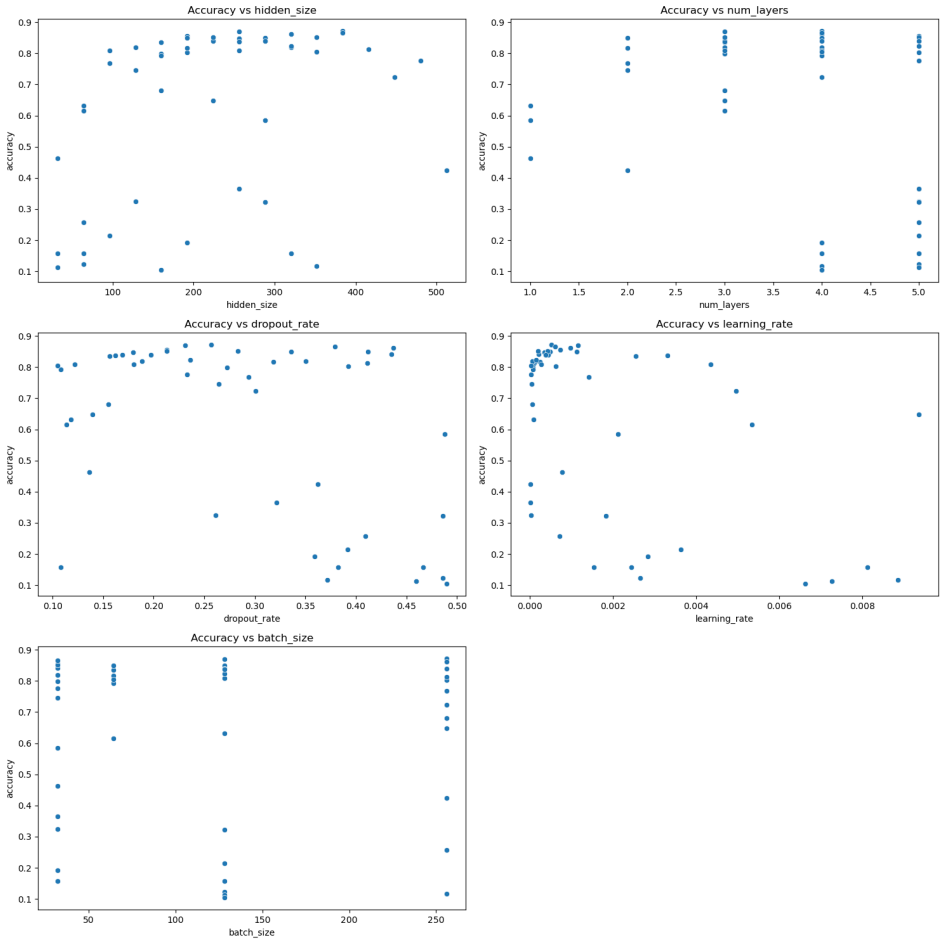

Figure 6: Relationship between Hyperparameters and Test Accuracy across Different Models

All layers maintain relatively small mean values close to zero (ranging from -0.014925 to 0.000694), indicating well-balanced weight distributions. The standard deviations gradually increase from the input layer (0.025017) to deeper layers (reaching 0.054369 in the output layer), suggesting broader weight distributions in deeper layers. This is visually confirmed by the distribution plots in Figure 7, which show approximately normal distributions with increasing spread in deeper layers. The weight ranges remain relatively consistent across hidden layers but expand in the output layer [-0.372, 0.139], as shown in both the statistics and the heatmaps in Figure 8.

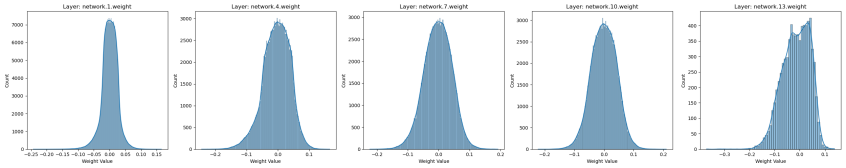

Figure 7: Weight Distribution Histograms Across Network Layers

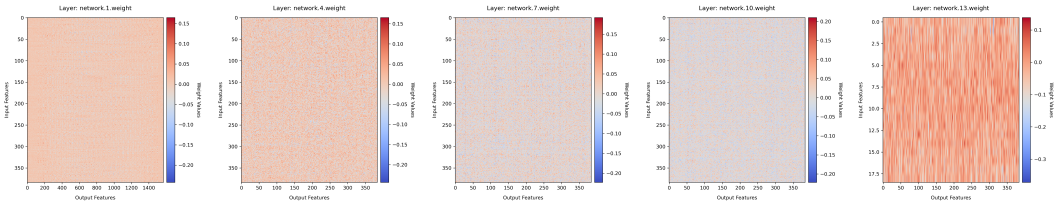

Figure 8: Weight Matrix Heatmaps of Network Layer

Performance Comparison between NN, RF and SVM

Implementation Analysis

Here, we implemented Random Forest (RF) and Support Vector Machine (SVM) using sklearn. The Random Forest classifier was configured with 100 estimators, and SVM utilized a Radial Basis Function (RBF) kernel, which was chosen for its ability to handle non-linear relationships in high-dimensional data.

Test Performance

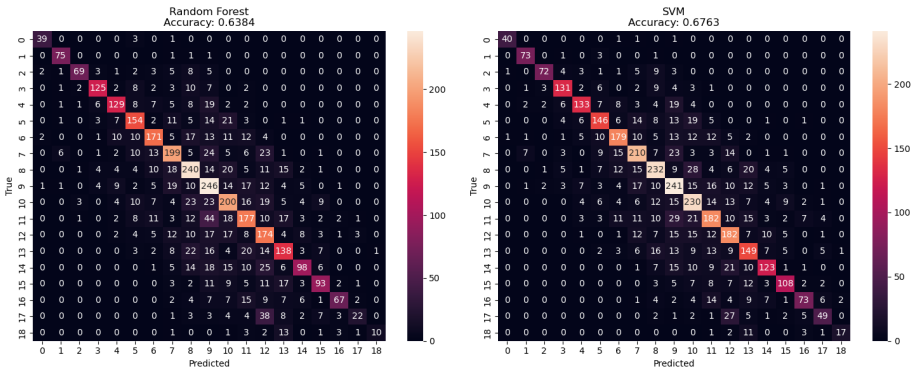

Figure 9: Confusion Matrix for RF and SVM

The performance comparison revealed distinct patterns across the three models. The Neural Network achieved the highest accuracy at 0.8716, substantially outperforming both traditional approaches. The SVM achieved an accuracy of 0.6763, performing slightly better than the Random Forest at 0.6384. The confusion matrices reveal several similar patterns in sum range performance: All models performed better with smaller sums (0,1) or larger sums (17,18), and performance degraded progressively for medium sums (5-13).

Weak Linear Results Comparison

Implementation Analysis

We implemented two distinct approaches using logistic regression as a weak classifier to investigate different strategies for handling the digit addition task:

1. Combined Classifier: This approach is a direct mapping from concatenated digit images to sum values. The implementation processes both digits simultaneously using a single logistic regression model trained on the full 1568-dimensional input (56×28 pixels)

2. Sequential Classifier: This approach decomposes the problem into individual digit recognition tasks. A single classifier is trained for digit recognition (stacking left and right images together), and use it for both left and right digits prediction. Then, the final sum is computed by adding the individual predictions. This model effectively reuses the same classifier for both digits, reducing the parameter count and leveraging digit-level patterns.

Both implementations used sklearn's LogisticRegression with maximum iteration 1000, and were evaluated across different training sample sizes (50, 100, 500, 1000) to assess their learning efficiency and scalability.

Results

The Sequential Classifier significantly outperformed the Combined Classifier across all training sample sizes. With just 50 training samples, it achieved 51% accuracy, and it reached 79% accuracy with 1000 samples. In contrast, the Combined Classifier showed notably lower performance, It started at 9% accuracy with 50 samples, and plateaued around 14% accuracy even with 1000 samples

These results suggest that decomposing the problem into individual digit recognition tasks (Sequential approach) provides several advantages:

- Better sample efficiency, achieving higher accuracy with fewer training examples.
- More stable learning curve with consistent improvements as sample size increases.

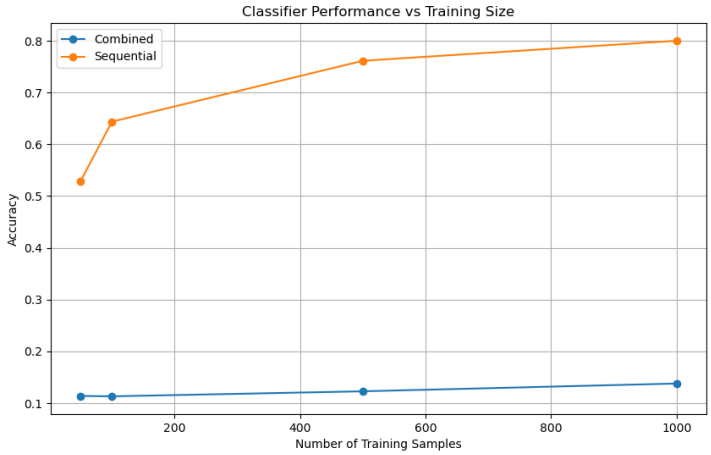

Figure 10: Accuracy Comparison between Sequential and Combined Classification Method

T-SNE distribution Results

Implementation Analysis

We implemented a t-SNE visualization pipeline to analyze both the raw input features and the learned representations from our best-performing neural network model. Here are some key points used in the implementation.

For the feature extraction, we used both direct flattened input images (1568 dimensional) and model embeddings, e.g. features from the penultimate layer of our neural network. Both inputs were standardized in separate StandardScaler instances.

For perplexity, we used an automatic search to optimize the perplexity from 30 to 70, with step size 5. We also used KL divergence as the optimization metric and found optimal perplexities of 65 for raw input and 70 for model embeddings.

Results Analysis

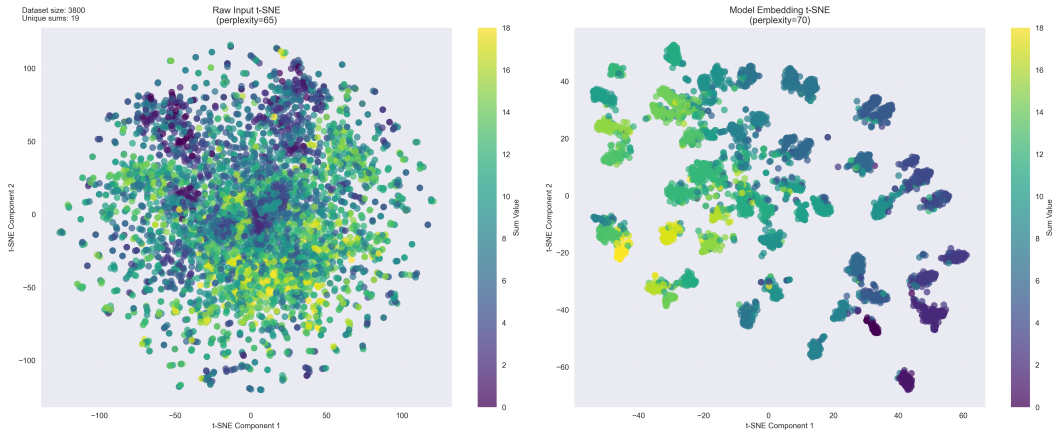

Figure 11: Visualization of T-SNE for Raw Input and Model Embedding

The resulting visualizations reveal a contrast between raw input and learned representations. The raw input t-SNE plot shows a largely uniform, circular distribution with limited clustering structure, suggesting that the pixel-level features provide little direct information about the addition results. In contrast, the model embedding visualization demonstrates clear clustering patterns, with similar sum values grouped together and a gradual transition between different sums. This contrast validates the effectiveness of our neural network's learning process.

Summary

Neural networks significantly outperform traditional machine learning approaches for this complex task, achieving 87.16% accuracy compared to 67.63% (SVM) and 63.84% (RF). Problem decomposition proves highly effective, as shown by the superior performance of the Sequential Classifier approach in the weak linear classifier analysis. The effectiveness of systematic hyperparameter optimization, which identified optimal model configurations. The importance of learned representations, as visualized through t-SNE analysis, showing how the model transforms raw pixel data into mathematically meaningful features.

Appendix

I declare that in the preparation of this coursework submission, autogeneration tools were used solely in a supportive capacity and not as a substitute for my own work. Specifically, used in development assistance:

- Initial project structure suggestions
- Documentation string templates
- Debugging support for Optuna hyperparameter optimization

All AI-generated suggestions were thoroughly reviewed and verified before incorporation. The core implementation work, including neural network architecture, data processing, and analysis, was completed independently I maintain comprehensive understanding of all components and can explain any part of the implementation in detail